

Classificação de arcos e componentes

Algoritmos e Estruturas de Dados 2

Prof. Dr. André Yoshiaki Kashiwabara

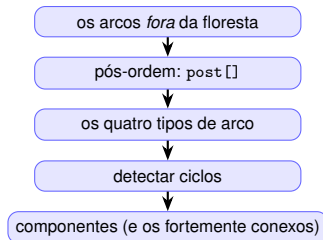
Departamento de Computação — Universidade Federal de Sergipe

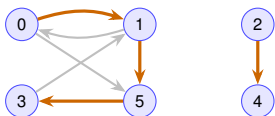
17 de setembro de 2026

Onde paramos

Da aula anterior

- `dfsR()` marca, avança enquanto pode e recua;
- `pre[v]` diz *se e quando* v foi descoberto;
- `GRAPHdfs()` varre tudo e produz uma **floresta** — os arcos por onde a busca avançou.





Laranja: os 4 arcos de árvore. Cinza: os 3 que ficaram de fora.

Os arcos cinzas não são lixo: cada um diz algo sobre a *estrutura* do grafo.

- 1-0 volta para um vértice que ainda está na pilha de recursão — fecha um **ciclo**;
- 0-5 aponta para um vértice que já é descendente de 0 — é um *atalho*;
- 3-1 também volta.

Precisamos de um critério mecânico para distinguir esses casos sem olhar o desenho.

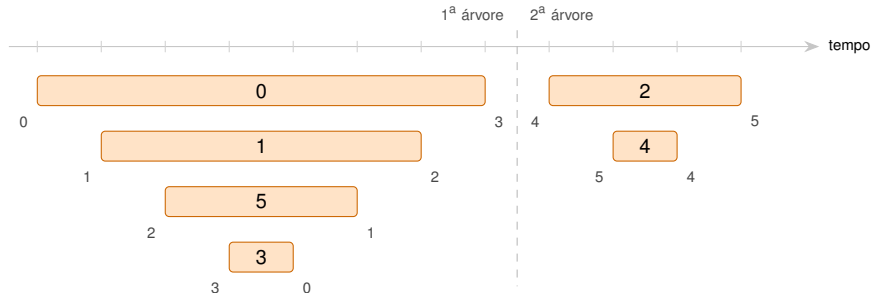
Pós-ordem

```
static int cnt, cnt1;
static int pre[maxV];    /* pre[v] = ordem de DESCOBERTA de v */
static int post[maxV];   /* post[v] = ordem de TERMINO de v */

static void dfsR( Graph G, vertex v) {
    pre[v] = cnt++;                          /* v abre: entrei em v */
    for (vertex w = 0; w < G->V; ++w)
        if (G->adj[v][w] != 0 && pre[w] == -1)
            dfsR( G, w);
    post[v] = cnt1++;                        /* v fecha: terminei v */
}
```

Uma linha nova, e ela muda tudo. `pre[]` é gravado na *entrada*; `post[]`, na *saída*. Entre os dois instantes, v está na pilha de recursão — dizemos que v está **aberto**.

Fonte: Feofiloff (2020), cap. *Busca em profundidade* [6]



Cada barra é o intervalo em que o vértice ficou **aberto**; à esquerda dela está **pre**, à direita **post**. Duas barras ou estão *encaixadas* (uma dentro da outra) ou são *disjuntas* — nunca se cruzam pela metade. Isso não é coincidência: é a pilha de recursão.

Fonte: Cormen et al. (2009), §22.3, fig. 22.5 [5]

Encaixe

Para quaisquer v e w , os intervalos de abertura de v e de w são disjuntos ou um contém o outro. E o intervalo de w está contido no de v *se e somente se* w é descendente de v na floresta DFS.

O critério que vamos usar o resto da aula

w é descendente de $v \iff \text{pre}[v] < \text{pre}[w] \text{ e } \text{post}[v] > \text{post}[w]$.

Por quê: v só fecha depois que todas as chamadas que ele disparou fecharam. Se v abre antes de w e ainda está aberto quando w abre, então w foi descoberto *dentro* da chamada $\text{dfsR}(v)$.

Fonte: Cormen et al. (2009), §22.3, Teorema 22.7 e Corolário 22.8 [5]; Feofiloff (2020) [6]

vértice v	0	1	2	3	4	5
pre[v]	0	1	4	3	5	2
post[v]	3	2	5	0	4	1

- Pré-ordem (ordem de pre): 0, 1, 5, 3, 2, 4 — a ordem em que os vértices foram *descobertos*.
- Pós-ordem (ordem de post): 3, 5, 1, 0, 4, 2 — a ordem em que foram *terminados*. Repare: as folhas terminam primeiro.
- Teste rápido: 3 é descendente de 0? $0 < 3$ e $3 > 0$. Sim. E 2 é descendente de 0? $\text{pre}[0] = 0 < 4$, mas $\text{post}[0] = 3 \not> 5$. Não.

Classificação de arcos

Os quatro tipos de arco



Seja $v-w$ um arco examinado durante a busca.

arco de árvore w ainda não tinha sido descoberto: a busca avançou por ele. Está na floresta.

arco de retorno w é *ancestral* de v (ou o próprio v): w ainda está aberto.

arco de avanço w é *descendente* de v , mas o arco não é de árvore: um atalho para baixo.

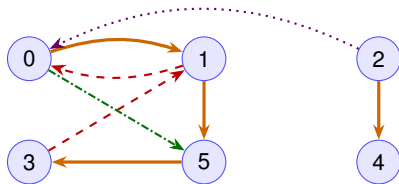
arco cruzado nem v é ancestral de w , nem w de v : w já fechou, em outro ramo ou em outra árvore.

A classificação depende da floresta, e portanto da ordem em que a busca visita os vértices. O que *não* depende é a existência de arco de retorno — voltaremos a isso.

Fonte: Cormen et al. (2009), §22.3 [5]; Sedgewick (2002), cap. 19 [18]; Ziviani (2011), §7.3 [23]

Um grafo com os quatro tipos

Ao grafo da aula acrescentamos o arco 2-0. Ele não muda `pre[]` nem `post[]` (2 só é visitado depois que 0 fechou), mas completa o catálogo.



— árvore 0-1 1-5 5-3 2-4 — retorno 1-0 3-1 — avanço 0-5 — cruzado 2-0

Tipo de $v-w$	Relação	pre	post
árvore / avanço	w descendente de v	$\text{pre}[v] < \text{pre}[w]$	$\text{post}[v] > \text{post}[w]$
retorno	w ancestral de v	$\text{pre}[w] < \text{pre}[v]$	$\text{post}[w] > \text{post}[v]$
cruzado	nenhum dos dois	$\text{pre}[w] < \text{pre}[v]$	$\text{post}[w] < \text{post}[v]$

- O quarto caso, $\text{pre}[w] > \text{pre}[v]$ com $\text{post}[w] > \text{post}[v]$, é **impossível**: os intervalos se cruzariam pela metade.
- Árvore e avanço têm a *mesma* assinatura em pre/post. Só o vetor $\text{pai}[]$ os separa: é de árvore quando $\text{pai}[w] = v$.

Fonte: Critério derivado do teorema dos parênteses — Cormen et al. (2009), §22.3 [5]

arco $v-w$	$pre[v]$	$post[v]$	$pre[w]$	$post[w]$	tipo
0-1	0	3	1	2	árvore
0-5	0	3	2	1	avanço (pai[5] = 1 $\neq$ 0)
1-0	1	2	0	3	retorno
1-5	1	2	2	1	árvore
2-0	4	5	0	3	cruzado
2-4	4	5	5	4	árvore
3-1	3	0	1	2	retorno
5-3	2	1	3	0	árvore

Leia a tabela linha a linha comparando com o critério do slide anterior — nenhum desenho é necessário.

```
/* Supoe que GRAPHdfs() ja preencheu pre[], post[] e pai[]. */
char *tipo( Graph G, vertex v, vertex w) {
    if (pre[v] < pre[w] && post[v] > post[w])          /* w descende de v */
        return (pai[w] == v) ? "arvore" : "avanco";
    if (pre[w] < pre[v] && post[w] > post[v])          /* w ancestral de v */
        return "retorno";
    return "cruzado";                                  /* nenhum dos dois */
}
```

Classificar durante a busca

Também dá para classificar *dentro* de `dfsR()`, sem esperar o fim, usando só `pre` e `post` parciais: `pre[w] == -1` $\Rightarrow$ árvore; `post[w] == -1` $\Rightarrow$ retorno (*w* ainda aberto); senão, `pre[w] > pre[v]` $\Rightarrow$ avanço, e `pre[w] < pre[v]` $\Rightarrow$ cruzado.

Fonte: Código no estilo de Feofiloff (2020), cap. *Classificação de arcos* [6]

Ciclos

Teorema

Um grafo dirigido tem um ciclo *se e somente se* qualquer execução de `GRAPHdfs()` produz pelo menos um arco de retorno.

- ($\Leftarrow$) Se $v-w$ é de retorno, w é ancestral de v : o caminho de w até v na floresta, seguido do arco $v-w$, é um ciclo.
- ($\Rightarrow$) Se há um ciclo, seja w o seu primeiro vértice descoberto pela busca. Todos os demais vértices do ciclo são alcançáveis a partir de w por vértices ainda brancos, logo são descendentes de w . O arco do ciclo que *entra* em w sai de um descendente de w : é de retorno.

Note a força do enunciado: o *número* e a *identidade* dos arcos de retorno dependem da ordem de visita, mas a *existência* não.

Fonte: Cormen et al. (2009), §22.3, Lema 22.11 [5]; Feofiloff (2020) [6]

```
static int temCiclo;
/* Tres estados: pre[v]==-1 (branco), post[v]==-1 (cinza), senao preto. */
static void dfsRciclo( Graph G, vertex v) {
    pre[v] = cnt++;                                /* v fica CINZA (aberto) */
    for (vertex w = 0; w < G->V; ++w)
        if (G->adj[v][w] != 0) {
            if (pre[w] == -1) dfsRciclo( G, w);        /* branco */
            else if (post[w] == -1) temCiclo = 1;      /* CINZA: retorno! */
                                                /* preto: ignore */
        }
    post[v] = cnt1++;                                /* v fica PRETO (fechado) */
}
```

No grafo da aula os arcos de retorno detectados são 1-0 e 3-1: há ciclo. Custo $\Theta(V + A)$ com listas — o mesmo da busca, sem nenhum passo extra.

Fonte: Esquema branco/cinza/preto: Cormen et al. (2009), §22.3 [5]

Errado

```
else if (pre[w] != -1) temCiclo = 1;
```

Isso acusa ciclo em *qualquer* arco de avanço ou cruzado.

Certo

```
else if (post[w] == -1) temCiclo = 1;
```

Só o vértice *aberto* — ainda na pilha — fecha ciclo.

Teste da diferença: o grafo com os arcos 0-1, 1-2 e 0-2 não tem ciclo algum, mas 0-2 é um arco de avanço e a versão errada o acusaria. Rode os dois no tutorial.

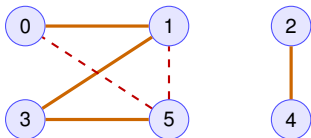
Grafos não dirigidos

Proposição

Em um grafo **não dirigido**, a busca em profundidade produz apenas arestas de **árvore** e de **retorno**.

- Cada aresta $\{v, w\}$ é examinada *duas vezes*, uma de cada ponta. Classifique-a na primeira vez; na segunda, w já estará fechado e a aresta já terá tipo.
- *Por que não há cruzada:* suponha que ao examinar $\{v, w\}$ o vértice w já esteja fechado e não seja ancestral de v . Então, quando a busca estava em w , ela examinou a aresta $\{w, v\}$; naquele momento v ainda era branco (pois v abriu depois), logo v teria virado descendente de w — contradição.
- O mesmo argumento elimina o arco de avanço.

Fonte: Cormen et al. (2009), §22.3, Teorema 22.10 [5]; Sedgewick e Wayne (2011), §4.1 [19]



Laranja cheio: árvore. Vermelho tracejado: retorno.
Nenhuma aresta de avanço ou cruzada.

Busca a partir de 0, vizinhos em ordem crescente.
Ordem de descoberta: 0, 1, 3, 5 e depois 2, 4.

v	0	1	2	3	4	5
pre	0	1	4	2	5	3
post	3	2	5	1	4	0

Árvore: $\{0, 1\}$, $\{1, 3\}$, $\{3, 5\}$, $\{2, 4\}$.

Retorno: $\{5, 0\}$ e $\{5, 1\}$ — cada uma fecha um ciclo.

```
static void dfsRcicloND( Graph G, vertex v, vertex pai_v) {
    pre[v] = cnt++;
    for (link a = G->adj[v]; a != NULL; a = a->next) {
        vertex w = a->w;
        if (pre[w] == -1)
            dfsRcicloND( G, w, v);
        else if (w != pai_v)           /* nao e a aresta por onde eu vim */
            temCiclo = 1;             /* aresta de retorno: ha ciclo */
    }
    post[v] = cnt1++;
}
```

Sem o teste $w \neq \text{pai}_v$, toda aresta $\{v, w\}$ seria acusada de ciclo, porque ela aparece nas duas listas de adjacência. A ressalva vale só para grafos *simples*: aresta paralela e laço são ciclos de verdade.

Componentes

Definição

Em um grafo não dirigido, v e w estão no mesmo **componente conexo** se existe caminho entre eles. Isso particiona os vértices: todo vértice está em exatamente um componente.

- Uma única busca a partir de s descobre exatamente o componente de s .
- Logo a varredura `GRAPHdfs()` descobre um componente por **árvore** da floresta: número de componentes = número de raízes.
- Basta rotular: $cc[v] = \text{número do componente de } v$.

A floresta depende da ordem das raízes; a *partição* em componentes, não. Rodar a busca a partir de 2 primeiro dá outra floresta e os mesmos dois componentes.

Fonte: Sedgewick e Wayne (2011), §4.1 [19]; Feofiloff, Kohayakawa e Wakabayashi (2011) [7]

```
static int cc[maxV];      /* cc[v] = numero do componente de v */

static void dfsRcc( Graph G, vertex v, int c) {
    cc[v] = c;
    for (link a = G->adj[v]; a != NULL; a = a->next)
        if (cc[a->w] == -1)
            dfsRcc( G, a->w, c);
}

int GRAPHcc( Graph G) {
    int c = 0;
    for (vertex v = 0; v < G->V; ++v) cc[v] = -1;
    for (vertex v = 0; v < G->V; ++v)
        if (cc[v] == -1) dfsRcc( G, v, c++);
    return c;
} /* numero de componentes */
```

cc[] substitui pre[] como marcador: um vetor só faz os dois papéis.

Fonte: Feofiloff (2020), cap. *Componentes* [6]; Sedgewick e Wayne (2011), §4.1 [19]

v	0	1	2	3	4	5
$cc[v]$	0	0	1	0	1	0

Dois componentes: $\{0, 1, 3, 5\}$ e $\{2, 4\}$.

Esse padrão — pré-processar uma vez, consultar muitas — vai reaparecer em caminhos mínimos e em árvores geradoras mínimas.

A ideia que se repete no curso

Gaste $\Theta(V + A)$ *uma vez* e depois responda “ v e w estão conectados?” em tempo **constante**: $cc[v] == cc[w]$.

Sem o pré-processamento, cada pergunta custaria uma busca inteira.

Nosso digrafo

A floresta DFS tem **2** árvores (raízes 0 e 2). Mas 2 alcança 4 e 4 não alcança 2: eles não são “equivalentes”.

Além disso, a partir de 4 a floresta teria 3 árvores.

O número de árvores depende da ordem — não serve como resposta.

Calcular essa partição em $\Theta(V + A)$ é possível — e usa exatamente a pós-ordem desta aula, aplicada ao grafo com os arcos invertidos. É o algoritmo de **Kosaraju**, o resto desta aula.

Fonte: Componentes fortemente conexos: Tarjan (1972) [21]; Sharir (1981) [20]; Cormen et al. (2009), §22.5 [5]

Componente fortemente conexo

$v \equiv w$ quando v alcança w e w alcança v . Isso sim é uma relação de equivalência, e a partição não depende de ordem alguma.

No nosso digrafo: $\{0, 1, 3, 5\}$, $\{2\}$ e $\{4\}$ — **três** componentes, não dois.

Componentes fortemente conexos

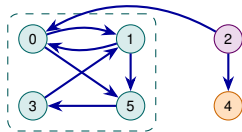
Definição

Em um digrafo, $v \equiv w$ quando existe caminho de v para w e de w para v . Um **componente fortemente conexo** é um conjunto *máximo* de vértices dois a dois equivalentes; o digrafo é **fortemente conexo** quando tem um componente só.

$\equiv$ é mesmo uma relação de equivalência: reflexiva (caminho de comprimento zero), simétrica (a definição já exige os dois sentidos) e transitiva (concatene os caminhos). Por isso os componentes **particionam** os vértices, e essa partição não depende de ordem de visita nenhuma.

Em grafo não dirigido a distinção não faz falta: lá “alcançar” já é simétrico.

Fonte: Feofiloff, Kohayakawa e Wakabayashi (2011) [7]; Cormen et al. (2009), §22.5 [5]



Três componentes: $\{0, 1, 3, 5\}$ (o ciclo $0 \rightarrow 5 \rightarrow 3 \rightarrow 1 \rightarrow 0$ passa por todos), $\{2\}$ e $\{4\}$: 2 alcança 0 e 4, mas ninguém volta até ele.

A estratégia

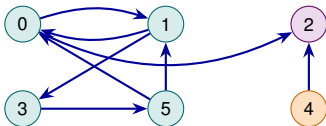
1. Rode `GRAPHdfs()` em G e guarde `post []`.
2. Construa G^R : o mesmo grafo com **todos os arcos invertidos**.
3. Rode a varredura em G^R , escolhendo as raízes em ordem **decrescente** de `post`. Cada árvore dessa segunda floresta é exatamente um componente fortemente conexo.

Nada além do que já temos: duas buscas em profundidade e a pós-ordem da primeira. Custo $\Theta(V + A)$, como uma busca — só que feita duas vezes.

Duas perguntas ficam em aberto: *por que inverter os arcos?* e *por que a pós-ordem?* Os próximos dois slides respondem uma de cada vez.

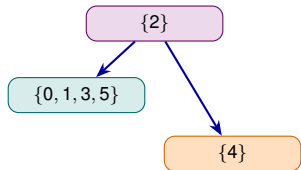
Fonte: Kosaraju (1978, inédito), publicado em Aho, Hopcroft e Ullman (1983), §6.7 [1]; Sharir (1981) [20]; Cormen et al. (2009), §22.5 [5]

G^R : os mesmos vértices, todos os arcos invertidos



As cores marcam os componentes fortes. v alcança w e w alcança v em $G \iff$ o mesmo vale em G^R — basta ler os dois caminhos ao contrário. Logo G e G^R têm os mesmos componentes.

A condensação de G : um vértice por componente



Ela é sempre um **DAG**: se houvesse ciclo entre dois componentes, ida e volta existiriam e os dois seriam um componente só.

Fonte: Condensação e conexidade forte: Cormen et al. (2009), §22.5, Lema 22.13 [5]; Feofiloff, Kohayakawa e Wakabayashi (2011) [7]

Lema

Se existe arco de um componente C para outro componente C' , então $\max_{v \in C} \text{post}[v] > \max_{w \in C'} \text{post}[w]$.

Prova. Não há caminho de C' de volta para C (a condensação é um DAG). Se a busca descobre primeiro um vértice de C , ela alcança todo C e todo C' por vértices brancos, e só fecha depois de todos eles. Se descobre primeiro um vértice de C' , essa busca nunca chega a C : todo C' fecha antes de qualquer vértice de C abrir. $\square$

Consequência

O vértice de maior post está num componente **fonte** da condensação. Em G^R esse componente vira **sumidouro**: a busca partindo dali *não consegue escapar* e descobre exatamente ele. Rotule, remova, repita — é o que a ordem decrescente faz, sem nunca construir a condensação.

Fonte: Cormen et al. (2009), §22.5, Lema 22.14 e Teorema 22.16 [5]; Sedgwick e Wayne (2011), §4.2 [19]

```
static int sc[maxV], ord[maxV];      /* sc[v]: componente forte de v */
/* dfsRsc(GR,v,c): a mesma dfsRcc() de antes, rotulando sc[] em GR */

int GRAPHscc( Graph G) {
    GRAPHdfs( G);                      /* 1: preenche post[] */
    Graph GR = GRAPHreverse( G);       /* 2: inverte os arcos */
    for (vertex v = 0; v < G->V; ++v)  /* ord: post decrescente */
        { ord[G->V-1 - post[v]] = v;   sc[v] = -1; }
    int c = 0;
    for (int i = 0; i < G->V; ++i)      /* 3: raizes em ord */
        if (sc[ord[i]] == -1) dfsRsc( GR, ord[i], c++);
    return c;                          /* qtd de componentes */
}
```

`post[]` é uma *permutação* de $0..V-1$: ordenar por post decrescente custa $\Theta(V)$, sem qsort.

Fonte: Código no estilo de Feofiloff (2020) [6]; algoritmo de Sharir (1981) [20]

v	0	1	2	3	4	5
post[v]	3	2	5	0	4	1
sc[v]	2	2	0	2	1	2

Ordem decrescente de post: 2, 4, 0, 1, 5, 3.

- raiz **2** em G^R : nada sai de 2 $\Rightarrow \{2\}$;
- raiz **4**: só o arco 4-2, e 2 já tem rótulo $\Rightarrow \{4\}$;
- raiz **0**: alcança 1, 3 e 5 $\Rightarrow \{0, 1, 3, 5\}$;
- 1, 5 e 3 já rotulados: termina com $c = 3$.

O que mudou

A floresta da *primeira* busca tinha 2 árvores e dependia da raiz. A da *segunda* tem 3 — e essa partição é a mesma para qualquer ordem inicial: quem a define é o grafo.

Brinde: os rótulos saem em ordem **topológica** da condensação. Aqui $\{2\}$ recebe o rótulo 0 e é justamente a fonte.

Fonte: Sedgewick e Wayne (2011), §4.2 [19]

Custo

Tarefa	Listas	Matriz
pre[] e post[]	$\Theta(V + A)$	$\Theta(V^2)$
classificar todos os arcos	$\Theta(V + A)$	$\Theta(V^2)$
decidir se há ciclo	$\Theta(V + A)$	$\Theta(V^2)$
rotular componentes (cc[])	$\Theta(V + A)$	$\Theta(V^2)$
componentes fortemente conexos (Kosaraju)	$\Theta(V + A)$	$\Theta(V^2)$
consultar “v e w conectados?” (depois)	$\Theta(1)$	$\Theta(1)$

- Nada aqui é mais caro que a própria busca: post[], o tipo do arco e o rótulo do componente saem “de graça” junto com ela.
- Kosaraju são *duas* buscas mais G^R : ainda $\Theta(V + A)$, com a constante dobrada.
- Espaço extra: $\Theta(V)$ nos vetores e $\Theta(V + A)$ para guardar G^R .

Fechamento

Exercícios

1. Acrescente `post[]` à sua `dfsR()` e reproduza a tabela `pre/post` desta aula.
2. Escreva `GRAPHclassify(G)`, que imprime cada arco do grafo com o seu tipo. Confira contra a tabela dos oito arcos.
3. Implemente `GRAPHcyclic(G)` com os três estados e mostre um grafo em que a versão “`pre[w] != -1`” erra.
4. Implemente `GRAPHcc(G)` e responda quantos componentes tem o grafo não dirigido do Exemplo A do tutorial.
5. Prove: um digrafo é acíclico sse nenhuma execução de `GRAPHdfs()` produz arco de retorno — e se trocarmos “nenhuma” por “alguma”?
6. Mostre um digrafo em que o número de árvores da floresta DFS muda conforme a raiz escolhida, mas o número de componentes fortemente conexos não.
7. Implemente `GRAPHreverse(G)` e `GRAPHscc(G)`; confira os três componentes do digrafo da aula e mostre o que quebra se as raízes forem tomadas em *post crescente*.

O tutorial traz esqueletos em C para os itens 2, 3, 4 e 7.

As ideias centrais

- `post []` custa uma linha e transforma a DFS em uma ferramenta de análise estrutural.
- Teorema dos parênteses: w descende de $v \iff \text{pre}[v] < \text{pre}[w]$ e $\text{post}[v] > \text{post}[w]$.
- Quatro tipos de arco em digrafo; sem direção, só árvore e retorno.
- Há ciclo $\iff$ há arco de retorno — “de retorno” é “o destino ainda está aberto”, não “já foi visto”.
- `cc []` particiona os vértices em $\Theta(V + A)$ e responde conexidade em $\Theta(1)$; em digrafo, vale perguntar por componentes *fortemente* conexos.
- Kosaraju: `post []` em G , depois DFS em G^R com raízes em `post` decrescente — cada árvore é um componente forte.

-  Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman.
Data Structures and Algorithms.
Addison-Wesley, 1983.
Seção 6.7: o algoritmo das duas buscas para componentes fortemente conexos, atribuído a S. R. Kosaraju (1978, inédito).
-  Jon Bentley.
Programming Pearls.
Addison-Wesley, 2 edition, 2000.
-  Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan.
Time bounds for selection.
Journal of Computer and System Sciences, 7(4):448–461, 1973.
Artigo original da mediana das medianas: seleção em $O(n)$ no pior caso.
-  J. Adrian Bondy and U. S. R. Murty.
Graph Theory, volume 244 of *Graduate Texts in Mathematics*.
Springer, 2008.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Paulo Feofiloff.
Algoritmos para grafos (em linguagem C).
Material didático online, DCC/IME-USP, 2020.
Capítulos *Grafos* e *Estruturas de dados para grafos*. Acesso em 01 set. 2026.



Paulo Feofiloff, Yoshiharu Kohayakawa, and Yoshiko Wakabayashi.
Uma introdução sucinta à teoria dos grafos.
Texto didático online, IME-USP, 2011.
Referência em português para as definições de teoria dos grafos. Acesso em 01 set. 2026.



Michael R. Garey and David S. Johnson.
Computers and Intractability: A Guide to the Theory of NP-Completeness.
W. H. Freeman, 1979.



David Gries.
The Science of Programming.
Springer-Verlag, 1981.



Michael Held and Richard M. Karp.
A dynamic programming approach to sequencing problems.
Journal of the Society for Industrial and Applied Mathematics, 10(1):196–210, 1962.



C. A. R. Hoare.
Algorithm 64: Quicksort.
Communications of the ACM, 4(7):321, 1961.



C. A. R. Hoare.
Algorithm 65: FIND.
Communications of the ACM, 4(7):321–322, 1961.
O quickselect, publicado por Hoare no mesmo ano do quicksort.



C. A. R. Hoare.

An axiomatic basis for computer programming.

Communications of the ACM, 12(10):576–580, 1969.



Donald E. Knuth.

The Art of Computer Programming, Volume 3: Sorting and Searching.

Addison-Wesley, 2 edition, 1998.

Seção 5.3.3: seleção mínima em número de comparações.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.

Pearson, 3 edition, 2012.



Udi Manber.

Introduction to Algorithms: A Creative Approach.

Addison-Wesley, 1989.



David R. Musser.

Introspective sorting and selection algorithms.

Software: Practice and Experience, 27(8):983–993, 1997.

O introselect: híbrido entre quickselect e mediana das medianas.



Robert Sedgewick.

Algorithms in C, Part 5: Graph Algorithms.

Addison-Wesley, 3 edition, 2002.

Fonte dos exemplos e exercícios do Capítulo 17 citados por Feofiloff.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4 edition, 2011.



Micha Sharir.

A strong-connectivity algorithm and its applications in data flow analysis.

Computers & Mathematics with Applications, 7(1):67–72, 1981.

Algoritmo das duas buscas (Kosaraju–Sharir) para componentes fortemente conexos.



Robert Tarjan.

Depth-first search and linear graph algorithms.

SIAM Journal on Computing, 1(2):146–160, 1972.

Artigo que estabeleceu a DFS como ferramenta algorítmica de tempo linear.



Nivio Ziviani.

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition, 2010.



Nivio Ziviani.

Projeto de Algoritmos com Implementações em Pascal e C.

Cengage Learning, 3 edition, 2011.

Capítulo 7: algoritmos em grafos.